RPL Set: RBS Ruleset (from MRM run)
Description: The way the spike and TBF overage rules are written, the EOWYST is reached at some point between Sept 1 and Sept 30. In low hydrologic years, the EOWYST will be reached slightly earlier in that month.
Notes:

Policy Group: 3 Cycle- Limits and Checks
Description: Cycle limits for safe channel capacity, USACE storage limits, etc. If the reservoir goes into a shortage, it is recorded here by way of the Shortage object. If that comes into play you must run Cycle 4.
Notes:

Rule: Shortage-KeepsNavajoWhole
Priority: 1
Description: This is the volume Navajo is shorted by. It's automatically added to Navajo as a "ghost" inflow. This is saved when we get to the end of Cycle 3 (by the next rule).
Notes:

```
Shortage.Local Inflow [] = IF ( Navajo Reservoir.Storage [ @"t - 1" ] < Navajo Reservoir.MinNIIPStorage [] ) THEN
                               VolumeToFlow ( Navajo Reservoir.MinNIIPStorage [] - Navajo Reservoir.Storage [ @"t - 1" ] , @"t" )
                           ELSE
                               0.00 "cfs"
                           END IF
```

Execute Only When:

```
GetRunCycleIndex () < 4.00 AND SWITCHES.Magic Water [] == 1.00
```

Rule: NIIPoffbelow5985
Priority: 2
Description:
Notes:

```
IF ( Navajo Reservoir.Pool Elevation [] < 5,985.00 "ft" ) THEN
    NIIP.Diversion [] = 0.00 "cfs"
END IF
```

Execute Only When:

```
GetRunCycleIndex () < 4.00
```

Rule: KnownReleaseOverwrite
Priority: 3
Description: Adds in recent releases that are known. Also can add in actual SPR when in the middle of the release, or some other SJRIP Adaptive Management Use. Leave NaN's when not changing anything and this will just overwrite.
Notes:

```
Navajo Reservoir.Release [] = IF ( NOT IsNaN NavajoData.KnownReleasePattern [] ) THEN
                                  NavajoData.KnownReleasePattern []
                              END IF
```

Execute Only When:

```
GetRunCycleIndex () == 3.00
```

Rule: ReleaseForUSACEStorageLimit
Priority: 4
Description:
Notes:

```
Navajo Reservoir.Release []
    = IF ( Navajo Reservoir.Storage [ @"t - 1" ] > NavajoData.NavFloodControlStorageLimit [] ) THEN
          Min ( VolumeToFlow ( Navajo Reservoir.Storage [ @"t - 1" ] - NavajoData.NavFloodControlStorageLimit [] , @"t" ) + Navajo Reservoir.Release [] ,
                MaxReleaseFunction_SafetyFactor () )
      END IF
```

Execute Only When:

```
GetRunCycleIndex () == 3.00 AND USACEWinterLimitSeason ()
```

Rule: USACEStorageLimit

Priority: 5
Description:
Notes:

```
NavajoData.NavFloodControlStorageLimit [ ]
    = IF ( NavajoData.USACEStorageLimitPercentage [ @"t - 1" ] < 0.95
           AND ( NavajoFloodControlLowerColumnValue ( ) == NavajoData.MaxStorage [ ]
                 AND NavajoFloodControlUpperColumnValue ( ) < NavajoFloodControlLowerColumnValue ( ) ) ) THEN
           NavajoData.MaxStorage [ ]
       ELSE
           NavajoData.FloodControlTables [ @"t" ,
                                           SumFlowsToVolume ( Navajo Reservoir.Inflow ,
                                                              @"t" ,
                                                              @"24:00:00 July 15, Current Year" ) ]
       END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 3.00 AND USACEWinterLimitSeason ( )
```

Rule: SafeChannelCapacity
Priority: 6
Description: If any of the flows below Farmington are >12,000, then reduce the release by (Flow - 12,000). Gets us pretty close.
Notes:

```
IF ( Max ( Max ( Max ( SJFS.Inflow [ @"t - 1" ] , SJFS.Outflow [ @"t - 1" ] ) , Max ( SJ4B.Outflow [ @"t - 1" ] , SJS4.Outflow [ @"t - 1" ] ) ) ) > 12,000.00 "cfs" ) THEN
    Navajo Reservoir.Release [ ] = Navajo Reservoir.Release [ ] - Max ( Max ( Max ( Max ( SJFS.Inflow [ @"t - 1" ] , SJFS.Outflow [ @"t - 1" ] ) , Max ( SJ4B.Outflow [ @"t - 1" ] , SJS4.Outflow [ @"t - 1" ] ) ) ) - 12,000.00 "cfs" , 0.00 "cfs" )
END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 3.00
```

Policy Group: 3 Cycle- Navajo 2016 Interim Operations
Description: The new Navajo rules are based on the new SJRIP plan of using all the AW down to 6050 to maximize the spring peak release days at 5,000 cfs. The release must be at least 21 days long and no longer than 60 days. There is no nose anymore- I take care of that using USACE flood control storage rules. This may or may not be changed in the future back to using a nose. The overage over 6063 is spread over summer TBF and finally a spike in the fall.
Notes:

Rule: DROA Water
Priority: 7
Description: For Chris and Wayne's flex analysis. Try moving
20kaf
50kaf
100 kaf
Until a shortage occurs.
Notes:

```
Navajo Reservoir.Release [ ] = Min ( Max ( Navajo Reservoir.Release [ ] + NavajoData.DROA [ ] , Navajo Reservoir.Release [ ] ) , MaxReleaseFunction_SafetyFactor ( ) )
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex ( ) == 3.00 AND SWITCHES.DROA [ ] == 1.00
```

Rule: AddSpike
Priority: 8
Description:
Notes:

```
Navajo Reservoir.Release [ ] = Min ( Max ( Navajo Reservoir.Release [ ] , NavajoData.SpikeRelease [ ] ) , MaxReleaseFunction_SafetyFactor ( ) )
```

Execute Only When:

```
GetRunCycleIndex ( ) == 3.00 AND NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND SpikeSeason ( )
```

Rule: CalculateSpike
Priority: 9
Description: Calculate spike volume for Sept 1- Sept 30 based on leftovers from nose and spike. This is a quick rule right now, should update to be more realistic (include correct ramping, etc.)
Notes:

```
NavajoData.SpikeRelease [ ] = IF ( Navajo Reservoir.Storage [ ] > ElevationToStorage ( Navajo Reservoir , 6,063.00 "feet" ) + 0.00 "acre-ft" ) THEN
                                  Min ( VolumeToFlow ( Navajo Reservoir.Storage [ ] - ( ElevationToStorage ( Navajo Reservoir , 6,063.00 "feet" ) + 0.00 "acre-ft" ) , @"t" ) , MaxReleaseFunction_SafetyFactor ( ) )
                                ELSE
                                  0.00 "cfs"
                                END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 3.00 AND NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND SpikeSeason ( )
```

Rule: JANLease Water
Priority: 10
Description: This is where we would add in JAN lease water as requested to the release. This water would flow downstream on top of the regular release. Is this correct? Would likely change year to year. If the water has not yet been requested in any one year, then this rule is turned off.
Notes:

```
Navajo Reservoir.Release [ ] = Min ( Max ( Navajo Reservoir.Release [ ] + Navajo Contracts.JANlease [ ] , Navajo Reservoir.Release [ ] ) , MaxReleaseFunction_SafetyFactor ( ) )
```

Execute Only When:

```
GetRunCycleIndex ( ) == 3.00 AND NOT HasRuleFiredSuccessfully ( "ThisRule" )
```

Rule: Manual Maintenance Release
Priority: 11
Description: When SPR has begun (when very close to SPR time) , the SPR must be manually input here-- this rule should be turned off and the SPR being used should be input into the "NavajoData.KnownReleasePattern" slot.
Notes:

```
Navajo Reservoir.Release [ ] = IF ( NOT IsNaN NavajoData.ManualMaintenanceRelease [ ] ) THEN
                                  NavajoData.ManualMaintenanceRelease [ ]
                                END IF
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex ( ) == 3.00
```

Rule: IncrementRelease_Simple
Priority: 12
Description: This rule increments the release into blocks (accounts for gate curve, min resolution of changes)
8/2/2018- updated to use "249cfs" instead of min release function
Notes:

```
Navajo Reservoir.Release [ ] = Min ( Ceiling ( Navajo Reservoir.Release [ ] , NavajoData.Release Increments [ ] ) , MaxReleaseFunction_SafetyFactor ( ) )
```

Execute Only When:

```
GetRunCycleIndex ( ) == 3.00 AND NOT HasRuleFiredSuccessfully ( "ThisRule" )
```

Rule: ReForecast Releases_SeriesTBF - Shortage
Priority: 13
Description: Forecasts the release required to meet the TBF minimum downstream plus the overage. If the function cannot find a release to use, hypsim will return a message in diagnostics and will be ineffective.

This rule fires in Cycle 3, as well as Cycle 2. The reason is just in case the SPR goes down since Cycle 2, we need to know what the release should be

to meet the TBF in those few days.

Notes:

```
WITH LIST forecastrelease = HypTargetSimWithStatus "BelowReservoirs" ,
                                                   SJAF.Inflow ,
                                                   NavajoData.SeriesMinRelease [] ,
                                                   NavajoData.MaxRelease [] ,
                                                   { { Florida River.Inflow , Florida River.Inflow [ @"t" ] , @"t" } ,
                                                     { Florida River.Inflow , Florida River.Inflow [ @"t" ] , @"t + 1" } ,
                                                     { Florida River.Inflow , Florida River.Inflow [ @"t" ] , @"t + 2" } ,
                                                     { Florida River.Inflow , Florida River.Inflow [ @"t" ] , @"t + 3" } ,
                                                     { Pine River.Inflow , Pine River.Inflow [ @"t" ] , @"t" } ,
                                                     { Pine River.Inflow , Pine River.Inflow [ @"t" ] , @"t + 1" } ,
                                                     { Pine River.Inflow , Pine River.Inflow [ @"t" ] , @"t + 2" } ,
                                                     { Pine River.Inflow , Pine River.Inflow [ @"t" ] , @"t + 3" } ,
                                                     { Basin Creek Y Animas.Inflow2 , Basin Creek Y Animas.Inflow2 [ @"t" ] , @"t" } ,
                                                     { Basin Creek Y Animas.Inflow2 , Basin Creek Y Animas.Inflow2 [ @"t" ] , @"t + 1" } ,
                                                     { Basin Creek Y Animas.Inflow2 , Basin Creek Y Animas.Inflow2 [ @"t" ] , @"t + 2" } ,
                                                     { Basin Creek Y Animas.Inflow2 , Basin Creek Y Animas.Inflow2 [ @"t" ] , @"t + 3" } ,
                                                   { { SJAF.Local Inflow , SJAF.Local Inflow [ @"t - 1" ] , @"t" } ,
                                                     { SJAF.Local Inflow , SJAF.Local Inflow [ @"t - 1" ] , @"t + 1" } ,
                                                     { SJAF.Local Inflow , SJAF.Local Inflow [ @"t - 1" ] , @"t + 2" } ,
                                                     { SJAF.Local Inflow , SJAF.Local Inflow [ @"t - 1" ] , @"t + 3" } ,
                                                     { SJFS.Local Inflow , SJFS.Local Inflow [ @"t - 1" ] , @"t" } ,
                                                     { SJFS.Local Inflow , SJFS.Local Inflow [ @"t - 1" ] , @"t + 1" } ,
                                                     { SJFS.Local Inflow ,
                                                                                                                                DO
```

```
                                        SJFS.Local Inflow [ @"t - 1" ] ,
                                        @"t + 2"
                                        SJFS.Local Inflow ,
                                        SJFS.Local Inflow [ @"t - 1" ] ,
                                        @"t + 3"
                                        SJS4.Local Inflow ,
                                        SJS4.Local Inflow [ @"t - 1" ] ,
                                        @"t"
                                        SJS4.Local Inflow ,
                                        SJS4.Local Inflow [ @"t - 1" ] ,
                                        @"t + 1"
                                        SJS4.Local Inflow ,
                                        SJS4.Local Inflow [ @"t - 1" ] ,
                                        @"t + 2"
                                        SJS4.Local Inflow ,
                                        SJS4.Local Inflow [ @"t - 1" ] ,
                                        @"t + 3"
                                        SJS4.Outflow ,
                                        @"t + 3" ,
                                        Min ( NavajoData.ShortageMinTBF [] + NavajoData.SummerTBFOverage [] ,
                                              NavajoData.MAX TBF [] ),
                                        10.00 "cfs" ,
                                        40.00 ,
                                        0.00
    NavajoData.ReleaseToMeetTBF [] = forecastrelease <1>
END WITH
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex () == 3.00 AND SWITCHES.Shortage [] == 1.00
```

Rule: ReForecast Releases
Priority: 14
Description: Forecasts the release required to meet the TBF minimum downstream plus the overage. If the function cannot find a release to use, hypsim will return a message in diagnostics and will be ineffective.

This rule fires in Cycle 3, as well as Cycle 2. The reason is just in case the SPR goes down since Cycle 2, we need to know what the release should be to meet the TBF in those few days.

Notes:

```
WITH LIST forecastrelease = HypTargetSimWithStatus "BelowReservoirs" ,                                  DO
                                                   SJAF.Inflow ,
                                                   NavajoData.SeriesMinRelease [] ,
                                                   NavajoData.MaxRelease [] ,
                                                   Florida River.Inflow ,
                                                   Florida River.Inflow [ @"t" ] ,
                                                   @"t"
                                                   Florida River.Inflow ,
                                                   Florida River.Inflow [ @"t" ] ,
                                                   @"t + 1"
                                                   Florida River.Inflow ,
                                                   Florida River.Inflow [ @"t" ] ,
                                                   @"t + 2"
                                                   Florida River.Inflow ,
                                                   Florida River.Inflow [ @"t" ] ,
                                                   @"t + 3"
                                                   Pine River.Inflow ,
                                                   Pine River.Inflow [ @"t" ] ,
                                                   @"t"
                                                   Pine River.Inflow ,
                                                   Pine River.Inflow [ @"t" ] ,
                                                   @"t + 1"
                                                   Pine River.Inflow ,
                                                   Pine River.Inflow [ @"t" ] ,
```

```
                                    ( @"t + 2" )
                                    ( Pine River.Inflow , )
                                    ( Pine River.Inflow [ @"t" ] , )
                                    ( @"t + 3" )
                                    ( Basin Creek Y Animas.Inflow2 , )
                                    ( Basin Creek Y Animas.Inflow2 [ @"t" ] , )
                                    ( @"t" )
                                    ( Basin Creek Y Animas.Inflow2 , )
                                    ( Basin Creek Y Animas.Inflow2 [ @"t" ] , )
                                    ( @"t + 1" )
                                    ( Basin Creek Y Animas.Inflow2 , )
                                    ( Basin Creek Y Animas.Inflow2 [ @"t" ] , )
                                    ( @"t + 2" )
                                    ( Basin Creek Y Animas.Inflow2 , )
                                    ( Basin Creek Y Animas.Inflow2 [ @"t" ] , )
                                    ( @"t + 3" )
                                    ( SJAF.Local Inflow , )
                                    ( SJAF.Local Inflow [ @"t - 1" ] , )
                                    ( @"t" )
                                    ( SJAF.Local Inflow , )
                                    ( SJAF.Local Inflow [ @"t - 1" ] , )
                                    ( @"t + 1" )
                                    ( SJAF.Local Inflow , )
                                    ( SJAF.Local Inflow [ @"t - 1" ] , )
                                    ( @"t + 2" )
                                    ( SJAF.Local Inflow , )
                                    ( SJAF.Local Inflow [ @"t - 1" ] , )
                                    ( @"t + 3" )
                                    ( SJFS.Local Inflow , )
                                    ( SJFS.Local Inflow [ @"t - 1" ] , )
                                    ( @"t" )
                                    ( SJFS.Local Inflow , )
                                    ( SJFS.Local Inflow [ @"t - 1" ] , )
                                    ( @"t + 1" )
                                    ( SJFS.Local Inflow , )
                                    ( SJFS.Local Inflow [ @"t - 1" ] , )
                                    ( @"t + 2" )
                                    ( SJFS.Local Inflow , )
                                    ( SJFS.Local Inflow [ @"t - 1" ] , )
                                    ( @"t + 3" )
                                    ( SJS4.Local Inflow , )
                                    ( SJS4.Local Inflow [ @"t - 1" ] , )
                                    ( @"t" )
                                    ( SJS4.Local Inflow , )
                                    ( SJS4.Local Inflow [ @"t - 1" ] , )
                                    ( @"t + 1" )
                                    ( SJS4.Local Inflow , )
                                    ( SJS4.Local Inflow [ @"t - 1" ] , )
                                    ( @"t + 2" )
                                    ( SJS4.Local Inflow , )
                                    ( SJS4.Local Inflow [ @"t - 1" ] , )
                                    ( @"t + 3" )
                                    SJS4.Outflow ,
                                    @"t + 3" ,
                                    Min ( NavajoData.MinTBF [] + NavajoData.SummerTBFOverage [] , )
                                        ( NavajoData.MAX TBF [] )
                                    10.00 "cfs" ,
                                    40.00 ,
                                    0.00
    NavajoData.ReleaseToMeetTBF [] = forecastrelease 〈1〉
END WITH
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex ( ) == 3.00 AND SWITCHES.Shortage [ ] == 0.00
```

Rule: CalculateOverageInCFS
Priority: 15
Description: This rule says what would I need to release today in order to have 0 AW left on Sept 30th? It's total outflow so I subtract NIIP. Not exact but gets closer and closer by Sept 30th.
Notes:

```
NavajoData.SummerTBFOverage [ ] = Max ( OutflowRequredToMeetTPE1 ( Navajo Reservoir ,
                                                                   Navajo Reservoir.Inflow ,
                                                                   Navajo Reservoir.Storage ,
                                                                   NavajoData.OpSpillEWYST ) - NIIP.Diversion [ ] ,
                                        0.00 "cfs" )
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex ( ) == 3.00 AND SummerBaseflowSeason ( )
```

Rule: InsertNewSPR
Priority: 16
Description:
Notes:

```
Navajo Reservoir.Release [ ] = IF ( NavajoData.Cycle3SpringPeakRelease [ ] > Navajo Reservoir.Release [ ] ) THEN
                                   Min ( NavajoData.Cycle3SpringPeakRelease [ ] , MaxReleaseFunction_SafetyFactor ( ) )
                               END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 3.00 AND NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND SpringPeakReleaseSeason ( )
```

Rule: SetCycle2OperationalRelease
Priority: 17
Description: Sets release as whatever the operational release from Cycle 2 was (to start)
Notes:

```
Navajo Reservoir.Release [ ] = NavajoData.Cycle2OperationalReleases [ ]
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex ( ) == 3.00
```

Rule: Count SPR Days
Priority: 18
Description: Count the days so we can subtract 350 cfs baseflow
Notes:

```
NavajoData.Cycle3NumSPRDays [ ] = IF ( NavajoData.Cycle3SpringPeakRelease [ ] > 4,599.00 "cfs" ) THEN
                                      1.00
                                  ELSE
                                      0.00
                                  END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 3.00 AND NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND SpringPeakReleaseSeason ( )
```

Rule: SetSPRbyVolume
Priority: 19
Description: this will pick the SPR based on volume. The 288,000 af is the equivalent to a 21-day SPR with 7-day ramp up and 14-day ramp down as rec by SJRIP. This number can be reduced to 0 af if we want to have the lower range releases as a possibility. SNB 2/2/23
Notes:

```
NavajoData.Cycle3SpringPeakRelease [ ] = IF ( SpringPeakReleaseSeason ( ) ) THEN
                                             IF ( NavajoData.Cycle3AvailableWater [ @"t" ] > NavajoData.MinSPRVolumeConsidered [ ] ) THEN
                                                 NavajoData.SPRTables Low Range_byVolume [ @"t" , NavajoData.Cycle3AvailableWater [ @"t" ] ]
                                             ELSE
                                                 NavajoData.SPRTables Low Range_byVolume [ @"t" , 0 "acre-feet" ]
```

```
            END IF
        END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 3.00 AND NOT HasRuleFiredSuccessfully ( "ThisRule" )
```

Rule: CalcMaxDaysAtPeak
Priority: 20
Description: This rule takes the AW and subtracts an appx 7-day ramp-up volume over base, and a 12-day ramp-down volume over base. Then it divdes the remainder by the volume of a single peak day over base. This returns the total number of days we will run at 5,000 cfs. In the end, it may be +/- a day or two.

Doesn't need to be a round number since we're using tables.

Change the 50,000 number if changing the # of ramping days, and point to the correct SPR ramp table.
3-day:18,000
7-day: 42,000
10-day: 61,000

Ramp-down (12-days): 65,000

Notes:

$$\text{NavajoData.Cycle3MaxDaysAtPeak}\,[\,] = \text{Max}\left(\text{Floor}\left(\frac{(\text{NavajoData.Cycle3AvailableWater}\,[\,] - 42{,}000.00\ \text{"acre-feet"} - 65{,}000.00\ \text{"acre-ft"})}{9{,}223.00\ \text{"acre-ft"}} - 1.00\ ,\ 1.00\right),\ 0.00\right)$$

Execute Only When:

```
GetRunCycleIndex ( ) == 3.00 AND NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND SpringPeakReleaseSeason ( )
```

Rule: AW Correction
Priority: 21
Description: When the resulting EOWST is not exactly 6050 ft, use this to adjust the AW calculation
Notes:

```
NavajoData.Cycle3AvailableWater [ ] = NavajoData.Cycle3AvailableWater [ ] - NavajoData.AWCorrection [ ]
```

Execute Only When:

```
GetRunCycleIndex ( ) == 3.00 AND NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND SpringPeakReleaseSeason ( )
```

Rule: Recalculated Available Water
Priority: 22
Description: AW calculation based on new calculation using EOWYST of 6050 ft. The AW calculated is the volume OVER base (350 cfs) that would be available.
Notes:

```
NavajoData.Cycle3AvailableWater [ ]
    = IF ( @"t" == FirstAWCalcDate ( ) ) THEN
        Navajo Reservoir.Storage [ @"t - 1" ]
        + SumFlowsToVolume ( Navajo Reservoir.Inflow ,
                             @"t" ,
                             @"24:00:00 September 30, Current Year" )
        - SumFlowsToVolume ( Navajo Reservoir.Diversion ,
                             @"t" ,
                             @"24:00:00 September 30, Current Year" )
        - SumSlot ( Navajo Reservoir.Evaporation ,
                    @"t" ,
                    @"24:00:00 September 30, Current Year" )
        - SumFlowsToVolume ( NavajoData.Cycle2OperationalReleases ,
                             @"t" ,
                             @"24:00:00 September 30, Current Year" )
        - ElevationToStorage ( Navajo Reservoir ,
                               NavajoData.AWEOWYST [ ] )
      ELSE
        NavajoData.Cycle3AvailableWater [ FirstAWCalcDate ( ) ]
      END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 3.00 AND NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND SpringPeakReleaseSeason ( )
```

Policy Group: 2 Cycle- Navajo Rules
Description: This is a "first pass " at operating Navajo. The purpose of this cycle is to first calculate what "Operational releases" are needed to satisfy the minimum TBF downstream. This must be calculated before the third cycle so that the Available Water for the year can be calculated. A set of "seed" spring peak releases are used (Manual Spring Peak Release) just to keep the pool elevation and storage at Navajo to something realistic, rather than allowing it to build up. For operational forecasts, it just uses last month's releases. At the end of this, the operational releases are stored for Cycle 3.
Notes:

Rule: SaveCycle2OperationalReleases
Priority: 23
Description: Operational releases are just saved for Cycle 2 calculations.

NaN function in there because there are a few NaN's at the very end of the run in ReleasetomeetTBF (3-day lag)
Notes:

```
NavajoData.Cycle2OperationalReleases [ ] = IF ( IsNaN NavajoData.ReleaseToMeetTBF [ ] ) THEN
                                               NavajoData.SeriesMinRelease [ ]
                                           ELSE
                                               NavajoData.ReleaseToMeetTBF [ ]
                                           END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 2.00 AND NOT HasRuleFiredSuccessfully ( "ThisRule" )
```

Rule: StoreCycle2Elevation
Priority: 24
Description:
Notes:

```
NavajoData.Cycle2NavajoElev [ ] = Navajo Reservoir.Pool Elevation [ ]
```

Execute Only When:

```
GetRunCycleIndex ( ) == 2.00 AND NOT HasRuleFiredSuccessfully ( "ThisRule" )
```

Rule: PickRelease
Priority: 25
Description: Sets release to the forecast releases. No rounding yet for this iteration.
Notes:

```
Navajo Reservoir.Release [ ] = Max ( NavajoData.ReleaseToMeetTBF [ ] , NavajoData.SeriesMinRelease [ ] )
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex ( ) == 2.00
```

Rule: Forecast Releases_SeriesTBF - Shortage
Priority: 26
Description: Forecasts the release required to meet the TBF minimum downstream. If the function cannot find a release to use, hypsim will return a message in diagnostics and will be ineffective. This is normal and expected during high flow releases like SPR or spike.
Notes:

```
WITH LIST forecastrelease = HypTargetSimWithStatus "BelowReservoirs" ,                                  DO
                                                   SJAF.Inflow ,
                                                   NavajoData.SeriesMinRelease [ ] ,
                                                   NavajoData.MaxRelease [ ] ,
                                                   ( { Florida River.Inflow ,          } ,          ) ,
                                                   ( { Florida River.Inflow [ @"t" ] , } ,          )
                                                   ( @"t"                               )
                                                   ( { Florida River.Inflow ,          } ,          )
                                                   ( { Florida River.Inflow [ @"t" ] , } ,          )
                                                   ( @"t + 1"                           )
                                                   ( { Florida River.Inflow ,          } ,          )
                                                   ( { Florida River.Inflow [ @"t" ] , } ,          )
                                                   ( @"t + 2"                           )
                                                   ( { Florida River.Inflow ,          } ,          )
                                                   ( { Florida River.Inflow [ @"t" ] , } ,          )
                                                   ( @"t + 3"                           )
```

```
{
  {
    {
      Pine River.Inflow ,
      Pine River.Inflow [ @"t" ] ,
      @"t"
    },
    {
      Pine River.Inflow ,
      Pine River.Inflow [ @"t" ] ,
      @"t + 1"
    },
    {
      Pine River.Inflow ,
      Pine River.Inflow [ @"t" ] ,
      @"t + 2"
    },
    {
      Pine River.Inflow ,
      Pine River.Inflow [ @"t" ] ,
      @"t + 3"
    },
    {
      Basin Creek Y Animas.Inflow2 ,
      Basin Creek Y Animas.Inflow2 [ @"t" ] ,
      @"t"
    },
    {
      Basin Creek Y Animas.Inflow2 ,
      Basin Creek Y Animas.Inflow2 [ @"t" ] ,
      @"t + 1"
    },
    {
      Basin Creek Y Animas.Inflow2 ,
      Basin Creek Y Animas.Inflow2 [ @"t" ] ,
      @"t + 2"
    },
    {
      Basin Creek Y Animas.Inflow2 ,
      Basin Creek Y Animas.Inflow2 [ @"t" ] ,
      @"t + 3"
    }
  },
  {
    {
      SJAF.Local Inflow ,
      SJAF.Local Inflow [ @"t - 1" ] ,
      @"t"
    },
    {
      SJAF.Local Inflow ,
      SJAF.Local Inflow [ @"t - 1" ] ,
      @"t + 1"
    },
    {
      SJAF.Local Inflow ,
      SJAF.Local Inflow [ @"t - 1" ] ,
      @"t + 2"
    },
    {
      SJAF.Local Inflow ,
      SJAF.Local Inflow [ @"t - 1" ] ,
      @"t + 3"
    },
    {
      SJFS.Local Inflow ,
      SJFS.Local Inflow [ @"t - 1" ] ,
      @"t"
    },
    {
      SJFS.Local Inflow ,
      SJFS.Local Inflow [ @"t - 1" ] ,
      @"t + 1"
    },
    {
      SJFS.Local Inflow ,
      SJFS.Local Inflow [ @"t - 1" ] ,
      @"t + 2"
    },
    {
      SJFS.Local Inflow ,
      SJFS.Local Inflow [ @"t - 1" ] ,
      @"t + 3"
    },
    {
      SJS4.Local Inflow ,
      SJS4.Local Inflow [ @"t - 1" ] ,
      @"t"
    },
    {
      SJS4.Local Inflow ,
      SJS4.Local Inflow [ @"t - 1" ] ,
      @"t + 1"
    },
    {
      SJS4.Local Inflow ,
      SJS4.Local Inflow [ @"t - 1" ] ,
      @"t + 2"
    },
    {
      SJS4.Local Inflow ,
      SJS4.Local Inflow [ @"t - 1" ] ,
      @"t + 3"
    }
  }
}
SJS4.Outflow ,
@"t + 3"
```

```
                                        NavajoData.ShortageMinTBF [] ,
                                        10.00 "cfs" ,
                                        40.00 ,
                                        0.00
    NavajoData.ReleaseToMeetTBF [] = forecastrelease <1>
END WITH
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex () == 2.00 AND SWITCHES.Shortage [] == 1.00
```

Rule: Forecast Releases
Priority: 27
Description: Forecasts the release required to meet the TBF minimum downstream. If the function cannot find a release to use, hypsim will return a message in diagnostics and will be ineffective.
Notes:

```
WITH LIST forecastrelease = HypTargetSimWithStatus "BelowReservoirs" ,                                  DO
                                        SJAF.Inflow ,
                                        NavajoData.SeriesMinRelease [] ,
                                        NavajoData.MaxRelease [] ,
                                        { { Florida River.Inflow , Florida River.Inflow [ @"t" ] , @"t" } ,
                                          { Florida River.Inflow , Florida River.Inflow [ @"t" ] , @"t + 1" } ,
                                          { Florida River.Inflow , Florida River.Inflow [ @"t" ] , @"t + 2" } ,
                                          { Florida River.Inflow , Florida River.Inflow [ @"t" ] , @"t + 3" } ,
                                          { Pine River.Inflow , Pine River.Inflow [ @"t" ] , @"t" } ,
                                          { Pine River.Inflow , Pine River.Inflow [ @"t" ] , @"t + 1" } ,
                                          { Pine River.Inflow , Pine River.Inflow [ @"t" ] , @"t + 2" } ,
                                          { Pine River.Inflow , Pine River.Inflow [ @"t" ] , @"t + 3" } ,
                                          { Basin Creek Y Animas.Inflow2 , Basin Creek Y Animas.Inflow2 [ @"t" ] , @"t" } ,
                                          { Basin Creek Y Animas.Inflow2 , Basin Creek Y Animas.Inflow2 [ @"t" ] , @"t + 1" } ,
                                          { Basin Creek Y Animas.Inflow2 , Basin Creek Y Animas.Inflow2 [ @"t" ] , @"t + 2" } ,
                                          { Basin Creek Y Animas.Inflow2 , Basin Creek Y Animas.Inflow2 [ @"t" ] , @"t + 3" } ,
                                        { { SJAF.Local Inflow , SJAF.Local Inflow [ @"t - 1" ] , @"t" } ,
                                          { SJAF.Local Inflow , SJAF.Local Inflow [ @"t - 1" ] , @"t + 1" } ,
                                          (
```

```
SJAF.Local Inflow ,
SJAF.Local Inflow [ @"t - 1" ] ,
@"t + 2"
SJAF.Local Inflow ,
SJAF.Local Inflow [ @"t - 1" ] ,
@"t + 3"
SJFS.Local Inflow ,
SJFS.Local Inflow [ @"t - 1" ] ,
@"t"
SJFS.Local Inflow ,
SJFS.Local Inflow [ @"t - 1" ] ,
@"t + 1"
SJFS.Local Inflow ,
SJFS.Local Inflow [ @"t - 1" ] ,
@"t + 2"
SJFS.Local Inflow ,
SJFS.Local Inflow [ @"t - 1" ] ,
@"t + 3"
SJS4.Local Inflow ,
SJS4.Local Inflow [ @"t - 1" ] ,
@"t"
SJS4.Local Inflow ,
SJS4.Local Inflow [ @"t - 1" ] ,
@"t + 1"
SJS4.Local Inflow ,
SJS4.Local Inflow [ @"t - 1" ] ,
@"t + 2"
SJS4.Local Inflow ,
SJS4.Local Inflow [ @"t - 1" ] ,
@"t + 3"
SJS4.Outflow ,
@"t + 3" ,
NavajoData.MinTBF [] ,
10.00 "cfs" ,
40.00 ,
0.00
```

```
NavajoData.ReleaseToMeetTBF [] = forecastrelease <1>
END WITH
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex () == 2.00 AND SWITCHES.Shortage [] == 0.00
```

Rule: SetMinRelease
Priority: 28
Description:
Notes:

```
Navajo Reservoir.Release [] = NavajoData.SeriesMinRelease []
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex () == 2.00
```

Rule: SetMinRelease for 2022-2024
Priority: 29
Description: This used to set min release by pool elevation, but now it sets min release based on if we are running min/most/max. Ideally the min release would either be consistent no matter what, or would be based on the forecast.
Notes:

```
NavajoData.SeriesMinRelease [] = 299.00 "cfs"
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex () == 2.00 AND @"t" <= @"24:00:00 December 31, 2023"
```

Rule: CalculateMinReleaseByElevation
Priority: 30
Description: This used to set min release by pool elevation, but now it sets min release based on if we are running min/most/max. Ideally the min release would either be consistent no matter what, or would be based on the forecast.

Typically we don't release as low as 250 cfs unless it looks like we are getting close to shortage, but 300 is a pretty normal min most of the time. I put 400 for the max probable.
Notes:

```
NavajoData.SeriesMinRelease [ ] = IF ( @"t" == @"Start Timestep" ) THEN
                                      299.00 "cfs"
                                  ELSE
                                      MinReleaseFunction ( )
                                  END IF
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex ( ) == 2.00
```

Rule: Set NVRN5 Inflow
Priority: 31
Description: According to UC and HDB calculation. Evap is included because it's int he change in storage calculation- and it needs to cancel out (mod unreg assumes the project isn't there, so evap wouldn't occur)
Notes:

```
Navajo Reservoir.Inflow [ ]
    = NavajoData.CBRFC Inflow [ ]
      - San Juan Chama Project.Diversion [ ]
      - VolumeToFlow ( Vallecito Reservoir.Storage [ ]
                       - Vallecito Reservoir.Storage [ @"t - 1" ] ,
                       @"t" )
      - VolumeToFlow ( Vallecito Reservoir.Evaporation [ ] ,
                       @"t" )
```

Execute Only When:

```
GetRunCycleIndex ( ) == 2.00
```

Policy Group: 1 Cycle- Nighthorse Rules
Description: Rules governing the operation of Ridges Basin and ALP
Notes:

Rule: MaxStorageSpill
Priority: 32
Description:
Notes:

```
Nighthorse Reservoir.Outflow [ ]
    = IF ( Nighthorse Reservoir.Storage [ ] > NighthorseData.MaxStorage [ ]
                                            + NighthorseData.StorageBuffer [ ] ) THEN
           VolumeToFlow ( Nighthorse Reservoir.Storage [ ]
                          - ( NighthorseData.MaxStorage [ ]
                            + NighthorseData.StorageBuffer [ ] ) ,
                          @"t" )
           + Nighthorse Reservoir.Outflow [ ]
       END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: MaxStorageStopPumping
Priority: 33
Description:
Notes:

```
IF ( Nighthorse Reservoir.Storage [ ] > NighthorseData.MaxStorage [ ] ) THEN
    Nighthorse Reservoir.Flow FROM Pumped Storage [ ] = 0.00 "cfs"
END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: DownstreamDemands
Priority: 34
Description:
Notes:

```
Nighthorse Reservoir.Outflow [ ] = NighthorseData.DownstreamDemand [ ]
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: PumpingLossToAnimas
Priority: 35
Description:
Notes:

```
Animas abv Nighthorse.Variable GainLoss [ ] = - 1.00 * Nighthorse Reservoir.Flow FROM Pumped Storage [ ]
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: PumpingOverride
Priority: 36
Description:
Notes:

```
Nighthorse Reservoir.Flow FROM Pumped Storage [ ] = IF ( NOT IsNaN NighthorseData.PumpOverride [ ] ) THEN
                                                        NighthorseData.PumpOverride [ ]
                                                    END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: PumpingLimitations
Priority: 37
Description:
Notes:

```
Nighthorse Reservoir.Flow FROM Pumped Storage [ ] = Min ( Nighthorse Reservoir.Flow FROM Pumped Storage [ ] ,
                                                          Min ( NighthorseData.MaxDPPPump [ ] ,
                                                                NighthorseData.Available To Pump [ ] ) )
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: PumpingOperation-withrefill-choose step pump THIS YEAR
Priority: 38
Description: If it's the first day of pumping, then we pick the appropriate step pump. Then anytime over the next two months that the reservoir is < 6882, the pumps kick back on and choose the May 1 pump rate. This will get messy once we get into actual pumping season, and we will need to rely on realtime pump estimates.
Notes:

```
Nighthorse Reservoir.Flow FROM Pumped Storage [ ] = IF ( IsFirstDayOfPumping ( ) ) THEN
                                                        NighthorseData.ManualPumpStep [ ]
                                                    ELSE
                                                        IF ( Nighthorse Reservoir.Pool Elevation [ @"t - 1" ] < NighthorseData.MaxElev [ ] ) THEN
                                                            NighthorseData.ManualPumpStep [ ]
                                                        ELSE
                                                            0.00 "cfs"
                                                        END IF
                                                    END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00 AND ALP Pumping Season ( ) AND NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND @"t" < @"24:00:00 December 31, 2023"
```

Rule: PumpingOperation-withrefill
Priority: 39
Description: If it's the first day of pumping, then we pick the appropriate step pump. Then anytime over the next two months that the reservoir is < 6882, the pumps kick back on and choose the May 1 pump rate. This will get messy once we get into actual pumping season, and we will need to rely on realtime pump estimates.
Notes:

```
Nighthorse Reservoir.Flow FROM Pumped Storage [ ] = IF ( IsFirstDayOfPumping ( ) ) THEN
                                                        PickStepPump ( )
                                                    ELSE
                                                        IF ( Nighthorse Reservoir.Pool Elevation [ @"t - 1" ] < NighthorseData.MaxElev [ ] ) THEN
                                                            Nighthorse Reservoir.Flow FROM Pumped Storage [ @"24:00:00 May 1, Current Year" ]
                                                        ELSE
                                                            0.00 "cfs"
                                                        END IF
                                                    END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00 AND ALP Pumping Season ( ) AND NOT HasRuleFiredSuccessfully ( "ThisRule" )
```

Rule: SetPumpsToZero
Priority: 40
Description:
Notes:

```
Nighthorse Reservoir.Flow FROM Pumped Storage [ ] = 0.00 "cfs"
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Policy Group: 1 Cycle- Lemon Rules
Description:
Notes:

Rule: MaxStorage
Priority: 41
Description:
Notes:

```
Lemon Reservoir.Outflow [ ] = IF ( Lemon Reservoir.Storage [ ] > LemonData.MaxStorage [ ] ) THEN
                                  VolumeToFlow ( Lemon Reservoir.Storage [ ] - LemonData.MaxStorage [ ] , @"t" ) + Lemon Reservoir.Outflow [ ]
                              END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: MaxSafeChannelRelease
Priority: 42
Description:
Notes:

```
Lemon Reservoir.Outflow [ ] = IF ( Lemon Reservoir.Outflow [ ] > LemonData.Safe Channel Capacity [ ] ) THEN
                                  LemonData.Safe Channel Capacity [ ]
                              END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: KnownRelease
Priority: 43
Description:
Notes:

```
Lemon Reservoir.Outflow [ ] = IF ( NOT IsNaN LemonData.KnownRelease [ ] ) THEN
```

```
                    LemonData.KnownRelease [ ]
                END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: Release Smoothing
Priority: 44
Description:
Notes:

```
Lemon Reservoir.Outflow [ ]
   = IF ( Lemon Reservoir.Outflow [ ] < Lemon Reservoir.Outflow [ @"t - 1" ]
          AND  Lemon Reservoir.Storage [ @"t - 1" ] / LemonData.FloodControlStorageLimit [ @"t - 1" ] >= LemonData.DownwardSmoothingACE_ResetValue [ ] ) THEN
          Lemon Reservoir.Outflow [ @"t - 1" ]
      END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: FloodControlLimits
Priority: 45
Description:
Notes:

```
Lemon Reservoir.Outflow [ ]
   = IF ( @"t" >= @"March 20" AND @"t" < @"July 11" AND Lemon Reservoir.Storage [ ] > LemonData.FloodControlStorageLimit [ ]
                                                        * LemonData.PercentofACE_Flood_Control_Limit [ ] ) THEN
          VolumeToFlow ( Lemon Reservoir.Storage [ ]
                         - ( LemonData.FloodControlStorageLimit [ ]
                             * LemonData.PercentofACE_Flood_Control_Limit [ ] ),
                         @"t" )
          + Lemon Reservoir.Outflow [ ]
      END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: SetFloodControlStorageLimit
Priority: 46
Description:
Notes:

```
LemonData.FloodControlStorageLimit [ ] = IF ( @"t" >= @"March 20" AND @"t" <= @"July 10" ) THEN
                                          TableInterpolation3D ( LemonData.Flood Control Table ,
                                                                 0.00 ,
                                                                 GetDayOfYear ( @"t" ) ,
                                                                 1.00 ,
                                                                 Remaining Spring Runoff Volume ( Lemon Reservoir ) ,
                                                                 2.00 ,
                                                                 @"t" )
                                       END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: IrrigationPondFilling
Priority: 47
Description: Stock pond filling over five days of 500 af on top of typical release (50 cfs/day plus 11 cfs/day = 61 cfs per day) Using 60 just because historically he doesn't seem to go above 60.
SNB Sept 2021
Notes:

```
[]    (                    ()                    []          )
```

```
Lemon Reservoir.Outflow [ ] = IF ( LemonPondFillingSeason ( ) AND Lemon Reservoir.Outflow [ ] < 50.00 "cfs" ) THEN
                                60.00 "cfs"
                            END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: SetRemainingSpringRunoff
Priority: 48
Description:
Notes:

```
LemonData.RemainingSpringRunoff [ ] = Remaining Spring Runoff Volume ( Lemon Reservoir )
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: MinEOYStorage
Priority: 49
Description:
Notes:

```
Lemon Reservoir.Outflow [ ] = IF ( Lemon Reservoir.Storage [ ] < LemonData.MinEOYStorage [ ] ) THEN
                                Min ( Lemon Reservoir.Inflow [ ] , LemonData.MedianDemands [ ] )
                            END IF
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex ( ) == 1.00 AND Lemon Irrigation Season ( )
```

Rule: Irrigation Release- NEW- no scaling, just ends
Priority: 50
Description:
Notes:

```
Lemon Reservoir.Outflow [ ] = IF ( Lemon Irrigation Season ( ) AND Lemon Reservoir.Outflow [ ] < LemonData.MedianDemands [ ] ) THEN
                                LemonData.MedianDemands [ ]
                            END IF
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex ( ) == 1.00
```

Rule: Irrigation Release
Priority: 51
Description:
Notes:

```
Lemon Reservoir.Outflow [ ] = IF ( Lemon Irrigation Season ( ) AND Lemon Reservoir.Outflow [ ] < LemonData.Downstream Demand [ ] ) THEN
                                LemonData.Downstream Demand [ ]
                            END IF
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex ( ) == 1.00
```

Rule: ShowDemandPercentages
Priority: 52
Description:
Notes:

$$\text{LemonData.DemandPercentages [ ] = Min} \left(1.00 , \frac{\text{SumFlowsToVolume (Lemon Reservoir.Outflow , @"24:00:00 May 1, Current Year" , @"t")}}{\text{SumFlowsToVolume (LemonData.MedianDemands , @"24:00:00 May 1, Current Year" , @"t")}} \right)$$

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00 AND NOT HasRuleFiredSuccessfully ( "Current Rule" ) AND @"t" == @"October 31"
```

Rule: SetDownstreamDemands
Priority: 53
Description:
Notes:

```
LemonData.Downstream Demand [ ]
    = IF ( @"t" == @"November 1" ) THEN
          LemonData.Downstream Demand [ @"t - 1" ]
      ELSE
          IF ( Lemon Irrigation Season ( ) ) THEN
              Max ( IF ( CalculateAvailableIrrigationVolume ( Lemon Reservoir ,
                                                              Lemon Reservoir.Inflow ,
                                                              Lemon Reservoir.Storage ,
                                                              LemonData.MinEOYStorage ,
                                                              Lemon Reservoir.Evaporation ) > LemonData.MedianDemandsVolRemaining [ ] ) THEN ,
                    LemonData.MedianDemands [ ]
                  ELSE
                    IF ( CalculateAvailableIrrigationVolume ( Lemon Reservoir ,
                                                              Lemon Reservoir.Inflow ,
                                                              Lemon Reservoir.Storage ,
                                                              LemonData.MinEOYStorage ,
                                                              Lemon Reservoir.Evaporation ) <= LemonData.MedianDemandsVolRemaining [ ] ) THEN
                        CalculateAvailableIrrigationVolume ( Lemon Reservoir ,
                                                             Lemon Reservoir.Inflow ,
                                                             Lemon Reservoir.Storage ,
                                                             LemonData.MinEOYStorage ,
                                                             Lemon Reservoir.Evaporation ) * LemonData.MedianDemands [ ]
                        ----------------------------------------------------------------------------------------------
                                  LemonData.MedianDemandsVolRemaining [ ]
                    ELSE
                        1.00 "cfs"
                    END IF
                  END IF
                  1.00 "cfs" )
          ELSE
              1.00 "cfs"
          END IF
      END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00 AND NOT HasRuleFiredSuccessfully ( "Current Rule" )
```

Rule: MinRelease_Winter
Priority: 54
Description:
Notes:

```
Lemon Reservoir.Outflow [ ] = 6.00 "cfs"
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex ( ) == 1.00 AND @"t" <= @"24:00:00 October 1, 2023"
```

Rule: Minimum Release
Priority: 55
Description:
Notes:

```
Lemon Reservoir.Outflow [ ] = LemonData.MinimumRelease [ ]
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Policy Group: 1 Cycle- Vallecito Rules
Description:
Notes:

Rule: KnownRelease
Priority: 56
Description:
Notes:

```
Vallecito Reservoir.Outflow [] = IF ( NOT IsNaN VallecitoData.KnownRelease [] ) THEN
                                    VallecitoData.KnownRelease []
                                 END IF
```

Execute Only When:

```
GetRunCycleIndex () == 1.00
```

Rule: MaxControlledRelease
Priority: 57
Description:
Notes:

```
Vallecito Reservoir.Outflow [] = IF ( Vallecito Reservoir.Outflow [] > VallecitoData.MaxOutletWorksRelease [] ) THEN
                                    VallecitoData.MaxOutletWorksRelease []
                                 END IF
```

Execute Only When:

```
GetRunCycleIndex () == 1.00
```

Rule: MaxStorageFill
Priority: 58
Description:
Notes:

```
Vallecito Reservoir.Outflow [] = IF ( Vallecito Reservoir.Storage [] > VallecitoData.MaxStorage [] ) THEN
                                    VolumeToFlow ( Vallecito Reservoir.Storage [] - VallecitoData.MaxStorage [] , @"t" ) + Vallecito Reservoir.Outflow []
                                 END IF
```

Execute Only When:

```
GetRunCycleIndex () == 1.00
```

Rule: MaxSafeChannelRelease
Priority: 59
Description:
Notes:

```
Vallecito Reservoir.Outflow [] = IF ( Vallecito Reservoir.Outflow [] > VallecitoData.Safe Channel Capacity [] ) THEN
                                    VallecitoData.Safe Channel Capacity []
                                 END IF
```

Execute Only When:

```
GetRunCycleIndex () == 1.00
```

Rule: ReleaseSmoothing
Priority: 60
Description:
Notes:

```
Vallecito Reservoir.Outflow []
   = IF ( Vallecito Reservoir.Outflow [] < Vallecito Reservoir.Outflow [ @"t - 1" ]                                  ) THEN
          AND  Vallecito Reservoir.Storage [ @"t - 1" ]
               ------------------------------------------------------------------------------------------------------
               VallecitoData.FloodControlStorageLimit [ @"t - 1" ]  >= VallecitoData.DownwardSmoothingACE_ResetValue [] )
          Vallecito Reservoir.Outflow [ @"t - 1" ]
     END IF
```

Execute Only When:

```
GetRunCycleIndex () == 1.00
```

Rule: FloodControlLimits
Priority: 61

Description:
Notes:

```
Vallecito Reservoir.Outflow [ ]
    = IF ( @"t" >= @"March 1" AND @"t" < @"August 1" AND Vallecito Reservoir.Storage [ ] > VallecitoData.FloodControlStorageLimit [ ]
                                                                                          * VallecitoData.PercentofACE_Flood_Control_Limit [ ] ) THEN
        VolumeToFlow ( Vallecito Reservoir.Storage [ ]
                       - ( VallecitoData.FloodControlStorageLimit [ ]
                         * VallecitoData.PercentofACE_Flood_Control_Limit [ ] ),
                       @"t" )
        + Vallecito Reservoir.Outflow [ ]
      END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: SetFloodControlStorageLimit
Priority: 62
Description:
Notes:

```
VallecitoData.FloodControlStorageLimit [ ] = IF ( @"t" >= @"March 1" AND @"t" <= @"July 31" ) THEN
                                                TableInterpolation3D ( VallecitoData.Flood Control Table ,
                                                                       0.00 ,
                                                                       GetDayOfYear ( @"t" ) ,
                                                                       1.00 ,
                                                                       Remaining Spring Runoff Volume ( Vallecito Reservoir ) ,
                                                                       2.00 ,
                                                                       @"t" )
                                              END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: IrrigationPondFilling
Priority: 63
Description: Using Lemon's pond filling season.
Notes:

```
Vallecito Reservoir.Outflow [ ] = IF ( LemonPondFillingSeason ( ) AND Vallecito Reservoir.Outflow [ ] < 100.00 "cfs" ) THEN
                                      100.00 "cfs"
                                  END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: SetRemainingSpringRunoff
Priority: 64
Description:
Notes:

```
VallecitoData.RemainingSpringRunoff [ ] = Remaining Spring Runoff Volume ( Vallecito Reservoir )
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: PostPeakSmoothing
Priority: 65
Description:
Notes:

```
Vallecito Reservoir.Outflow [ ]
    = IF ( PostPeakSmoothingPeriod ( ) ) THEN
        Max ( VolumeToFlow ( ( ( Vallecito Reservoir.Storage [ ]
                               + FlowToVolume ( Vallecito Reservoir.Outflow [ ] , ) ) ) , ) ,
```

```
                    ( ( ( @"t" ) )
                    + Remaining Post Peak Inflow Volume ( )
                    - VallecitoData.MaxWinterStorage [ ] )
                    ( GetJulianDate ( @"24:00:00 November 1, Current Year" )
                    - GetJulianDate ( @"t" ) )
                    @"t" )
                    ) )
                    Vallecito Reservoir.Outflow [ ] )
        END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: Deadpool
Priority: 66
Description:
Notes:

```
Vallecito Reservoir.Outflow [ ] = IF ( Vallecito Reservoir.Storage [ ] < VallecitoData.MinEOYStorage [ ] ) THEN
                                      VallecitoData.MinimumRelease [ ]
                                  END IF
```

Execute Only When:

```
TRUE
```

Rule: Irrigation Release
Priority: 67
Description:
Notes:

```
Vallecito Reservoir.Outflow [ ] = IF ( Vallecito Irrigation Season ( ) AND Vallecito Reservoir.Outflow [ ] < VallecitoData.Downstream Demand [ ] ) THEN
                                      VallecitoData.Downstream Demand [ ]
                                  END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00 AND NOT HasRuleFiredSuccessfully ( "Current Rule" )
```

Rule: ShowDemandPercentages
Priority: 68
Description:
Notes:

```
VallecitoData.DemandPercentages [ ] = Min ( 1.00 , SumFlowsToVolume ( Vallecito Reservoir.Outflow , @"24:00:00 May 9, Current Year" , @"t" )
                                                   SumFlowsToVolume ( VallecitoData.MedianDemands , @"24:00:00 May 9, Current Year" , @"t" ) )
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00 AND NOT HasRuleFiredSuccessfully ( "Current Rule" ) AND @"t" == @"October 31"
```

Rule: SetDownstreamDemands
Priority: 69
Description:
Notes:

```
VallecitoData.Downstream Demand [ ]
    = IF ( @"t" == @"November 1" ) THEN
          VallecitoData.Downstream Demand [ @"t - 1" ]
      ELSE
          IF ( Vallecito Irrigation Season ( ) ) THEN
              Max ( IF ( CalculateAvailableIrrigationVolume ( Vallecito Reservoir ,
                                                              Vallecito Reservoir.Inflow ,
                                                              Vallecito Reservoir.Storage ,
                                                              VallecitoData.MinEOYStorage ,
                                                              Vallecito Reservoir.Evaporation ) > VallecitoData.MedianDemandsVolRemaining [ ] ) THEN ,
                    VallecitoData.MedianDemands [ ]
                  ELSE
                      ( ( ( \ [ ] )
```

```
IF ( CalculateAvailableIrrigationVolume ( Vallecito Reservoir ,
                                          Vallecito Reservoir.Inflow ,
                                          Vallecito Reservoir.Storage ,
                                          VallecitoData.MinEOYStorage ,
                                          Vallecito Reservoir.Evaporation ) <= VallecitoData.MedianDemandsVolRemaining [ ] ) THEN
    CalculateAvailableIrrigationVolume ( Vallecito Reservoir ,
                                         Vallecito Reservoir.Inflow ,
                                         Vallecito Reservoir.Storage ,
                                         VallecitoData.MinEOYStorage ,
                                         Vallecito Reservoir.Evaporation ) * VallecitoData.MedianDemands [ ]
    ------------------------------------------------------------------------------------------------------
                    VallecitoData.MedianDemandsVolRemaining [ ]
ELSE
    1.00 "cfs"
END IF
END IF
1.00 "cfs"
ELSE
    1.00 "cfs"
END IF
END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00 AND NOT HasRuleFiredSuccessfully ( "Current Rule" )
```

Rule: WinterStorageLimit
Priority: 70
Description: Set release such that reservoir storage is less than or equal to 77,000 af. This is to keep reservoir elevation below the bottom of radial gates during icing season.
Notes:

```
Vallecito Reservoir.Outflow [ ]
    = IF ( Winter Storage Season ( ) AND Vallecito Reservoir.Storage [ ] > VallecitoData.MaxWinterStorage [ ] ) THEN
        Max ( VolumeToFlow ( Vallecito Reservoir.Storage [ ] + FlowToVolume ( Vallecito Reservoir.Outflow [ ] , @"t" ) - VallecitoData.MaxWinterStorage [ ] , @"t" ) ,
              Vallecito Reservoir.Outflow [ ] )
    END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: SetWinterRelease
Priority: 71
Description:
Notes:

```
Vallecito Reservoir.Outflow [ ]
    = IF ( Winter Storage Season ( ) ) THEN
        Max ( VolumeToFlow ( ( ( Vallecito Reservoir.Storage [ @"t - 1" ]
                               + Remaining Winter Inflow Volume ( Vallecito Reservoir ) )
                               - VallecitoData.MaxWinterStorage [ ] )
                              ------------------------------------------------------
                                        DaysUntilEndOfWinterStorage ( )
                              @"t" ) ,
              Vallecito Reservoir.Outflow [ ] )
    END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Rule: SetMinReleaseThisYear
Priority: 72
Description:
Notes:

```
Vallecito Reservoir.Outflow [] = 5.00 "cfs"
```

Execute Only When:

```
GetRunCycleIndex () == 1.00 AND @"t" < @"24:00:00 June 1, 2023"
```

Rule: Minimum Release
Priority: 73
Description:
Notes:

```
Vallecito Reservoir.Outflow [] = VallecitoData.MinimumRelease []
```

Execute Only When:

```
GetRunCycleIndex () == 1.00
```

Policy Group: 1 Cycle- Set Losses
Description:
Notes:

Rule: Set Local Inflows
Priority: 74
Description: Daily gainloss is a calculated gage-to-gage statistical loss. Choose statistical for entire model run in SWITCHES object.
Local inflows will never make river go less than 0. This is recorded in the reach "unidentified loss".
Yes.
Notes:

```
Animas River.Local Inflow [] = Animas River.DailyGainLoss [ @"t" , SWITCHES.ReachLossPercentile [] ]
SJAF.Local Inflow [] = SJAF.DailyGainLoss [ @"t" , SWITCHES.ReachLossPercentile [] ]
SJFS.Local Inflow [] = SJFS.DailyGainLoss [ @"t" , SWITCHES.ReachLossPercentile [] ]
SJS4.Local Inflow [] = SJS4.DailyGainLoss [ @"t" , SWITCHES.ReachLossPercentile [] ]
SJ4B.Local Inflow [] = SJ4B.DailyGainLoss [ @"t" , SWITCHES.ReachLossPercentile [] ]
Pine River.Local Inflow [] = Pine River.DailyGainLoss [ @"t" , SWITCHES.ReachLossPercentile [] ]
Florida River.Local Inflow [] = Florida River.DailyGainLoss [ @"t" , SWITCHES.ReachLossPercentile [] ]
```

Execute Only When:

```
GetRunCycleIndex () == 1.00 AND SWITCHES.UseNewGainLosses [] == 0.00
```

Rule: Set Local Inflows- Original
Priority: 75
Description: Daily gainloss is a calculated gage-to-gage statistical loss. Choose statistical for entire model run in SWITCHES object.
Local inflows will never make river go less than 0. This is recorded in the reach "unidentified loss".
Notes:

```
Animas River.Local Inflow [] = Animas River.DailyGainLoss [ @"t" , SWITCHES.ReachLossPercentile [] ]
SJAF.Local Inflow [] = SJAF.DailyGainLoss [ @"t" , SWITCHES.ReachLossPercentile [] ]
SJFS.Local Inflow [] = SJFS.DailyGainLoss [ @"t" , SWITCHES.ReachLossPercentile [] ]
SJS4.Local Inflow [] = SJS4.DailyGainLoss [ @"t" , SWITCHES.ReachLossPercentile [] ]
SJ4B.Local Inflow [] = SJ4B.DailyGainLoss [ @"t" , SWITCHES.ReachLossPercentile [] ]
Pine River.Local Inflow [] = Pine River.DailyGainLoss [ @"t" , SWITCHES.ReachLossPercentile [] ]
Florida River.Local Inflow [] = Florida River.DailyGainLoss [ @"t" , SWITCHES.ReachLossPercentile [] ]
```

Execute Only When:

```
GetRunCycleIndex () == 1.00 AND SWITCHES.UseNewGainLosses [] == 1.00
```

Rule: Set Local Inflows- Forecast Based
Priority: 76
Description: Daily gainloss is a calculated gage-to-gage statistical loss. Choose statistical for entire model run in SWITCHES object.
Local inflows will never make river go less than 0. This is recorded in the reach "unidentified loss".
Notes:

```
# Use DRGC2 for forecast
Animas River.Local Inflow [] = Animas River.DailyGainLoss [ @"t" , Lookup Loss Percentile ( "DRGC2" ) ]
# Use NVRN5
```

```
SJAF.Local Inflow [] = SJAF.DailyGainLoss [ @"t" , Lookup Loss Percentile ( "NVRN5" ) ]
SJFS.Local Inflow [] = SJFS.DailyGainLoss [ @"t" , Lookup Loss Percentile ( "NVRN5" ) ]
SJS4.Local Inflow [] = SJS4.DailyGainLoss [ @"t" , Lookup Loss Percentile ( "NVRN5" ) ]
SJ4B.Local Inflow [] = SJ4B.DailyGainLoss [ @"t" , Lookup Loss Percentile ( "NVRN5" ) ]
# Use LEMC2 for the forecast location
Pine River.Local Inflow [] = Pine River.DailyGainLoss [ @"t" , Lookup Loss Percentile ( "VCRC2" ) ]
# Use VCRC2 for the forecast location
Florida River.Local Inflow [] = Florida River.DailyGainLoss [ @"t" , Lookup Loss Percentile ( "LEMC2" ) ]
# Write out the percentile used
GainLossCalculations.ForecastPercentileNVRN5 [] = Lookup Loss Percentile ( "NVRN5" )
GainLossCalculations.ForecastPercentileDRGC2 [] = Lookup Loss Percentile ( "DRGC2" )
GainLossCalculations.ForecastPercentileVCRC2 [] = Lookup Loss Percentile ( "VCRC2" )
GainLossCalculations.ForecastPercentileLEMC2 [] = Lookup Loss Percentile ( "LEMC2" )
```

Execute Only When:

```
GetRunCycleIndex () == 1.00 AND SWITCHES.UseNewGainLosses [] == 2.00
```

Rule: SetShortageLocalInflow
Priority: 77
Description: This will bound the Animas Losses between a max of 775 cfs and a min of -500 cfs. Based on LOSSES_V.xlsm.
This is just a max and min observed for the whole year 1999-2013. Next step would be put into a table by month of year.
Notes:

```
Shortage.Local Inflow [] = 0.00 "cfs"
```

Execute Only When:

```
GetRunCycleIndex () == 1.00
```

Rule: Set Shortage Inflow
Priority: 78
Description:
Notes:

```
Shortage.Inflow [] = 0.00 "cfs"
```

Execute Only When:

```
GetRunCycleIndex () == 1.00
```

Rule: Set SJC Diversions
Priority: 79
Description:
Notes:

```
San Juan Chama Project.Diversion [] = San Juan Chama Project.Diversion Requested []
```

Execute Only When:

```
GetRunCycleIndex () == 1.00
```

Rule: Set NIIP Diversions
Priority: 80
Description:
Notes:

```
NIIP.Diversion [] = NIIP.Diversion Requested []
```

Execute Only When:

```
GetRunCycleIndex () == 1.00
```

Rule: Manual NIIP_Shortage
Priority: 81
Description: Reduces NIIP diversion during a shortage in the timeframe specified. Use proportional estimates to reduce NIIP by an exact amount. THen resolve the remainder through the downstream release.
Notes:

```
NIIP.Diversion Requested [ ] = NIIP.StaticYearlyNIIPvalues_SHORTAGE [ ]
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00 AND SWITCHES.Shortage [ ] == 1.00
```

Rule: MANUAL NIIP
Priority: 82
Description:
Notes:

```
NIIP.Diversion Requested [ ] = NIIP.StaticYearlyNIIPvalues [ ]
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00
```

Policy Group: 1 Cycle - OFFICIAL INFLOW FORECASTS
Description:
Notes:

Rule: Set SJC Diversion
Priority: 83
Description: Daily SJC diversion is directly related to the daily MUI flows. This correlation was made by comparing historic Azotea flows and MUI flows from 10/1/1990 - 8/2/2022 We are assuming on a daily basis that Azotea will pull whatever is available. up to max of 950 cfs C:\Users\sbehery\OneDrive - DOI\RIVERWARE MODELS\SAN JUAN BASIN OPERATIONAL MODEL\San Juan Chama.csv
Notes:

```
FOR ( DATETIME date IN @"Start Timestep" TO @"Finish Timestep" ) DO
    San Juan Chama Project.Diversion Requested [ date ] = Max ( Min ( ( NavajoData.CBRFC Inflow [ date ] - 419.40 "cfs" ) * 1.00 "cfs" , 950.00 "cfs" ) / 6.61 "cfs" , 0.00 "cfs" )
END FOR
```

Execute Only When:

```
@"t" == @"Start Timestep" AND GetRunCycleIndex ( ) == 1.00 AND SWITCHES.MODE [ ] == 0.00
```

Rule: Set Month 1 Inflows
Priority: 84
Description: Do not use execution constraints. Inflows must be input every cycle (why?) or model will not execute completely.
Notes:

```
Vallecito Reservoir.Inflow [ ] = IF ( NOT IsNaN INFLOW SO FAR THIS MONTH.Vallecito [ ] ) THEN
                                     INFLOW SO FAR THIS MONTH.Vallecito [ ]
                                 END IF
Lemon Reservoir.Inflow [ ] = IF ( NOT IsNaN INFLOW SO FAR THIS MONTH.Lemon [ ] ) THEN
                                 INFLOW SO FAR THIS MONTH.Lemon [ ]
                             END IF
NavajoData.CBRFC Inflow [ ] = IF ( NOT IsNaN INFLOW SO FAR THIS MONTH.Navajo [ ] ) THEN
                                     INFLOW SO FAR THIS MONTH.Navajo [ ]
                                 END IF
Animas abv Nighthorse.Inflow [ ] = IF ( NOT IsNaN INFLOW SO FAR THIS MONTH.AnimasDur [ ] ) THEN
                                         INFLOW SO FAR THIS MONTH.AnimasDur [ ]
                                     END IF
```

Execute Only When:

```
( NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex ( ) == 1.00 ) AND SWITCHES.MODE [ ] == 0.00
```

Rule: Set Inflows
Priority: 85
Description: Do not use execution constraints. Inflows must be input every cycle (why?) or model will not execute completely.
Notes:

```
FOR ( DATETIME date IN @"Start Timestep" TO @"Finish Timestep" ) DO
    Vallecito Reservoir.Inflow [ date ] = InflowDisagg.InflowForecastVallecito [ date ]
    NavajoData.CBRFC Inflow [ date ] = InflowDisagg.InflowForecastNavajo [ date ]
    Lemon Reservoir.Inflow [ date ] = InflowDisagg.InflowForecastLemon [ date ]
```

```
    Animas abv Nighthorse.Inflow [ date ] = InflowDisagg.InflowForecastAnimas [ date ]
END FOR
```

Execute Only When:

```
@"t" == @"Start Timestep" AND ( NOT HasRuleFiredSuccessfully ( "ThisRule" ) AND GetRunCycleIndex ( ) == 1.00 ) AND SWITCHES.MODE [ ] == 0.00
```

Rule: 1 Cycle - Write Scaled Forecast
Priority: 86
Description:
Notes:

```
FOR ( STRING forecastLocation IN { "Vallecito" , "Lemon" , "Animas" , "Navajo" } ) DO
    # Loop over all daily timesteps in the run.
    FOR ( DATETIME date IN @"Start Timestep" TO @"Finish Timestep" ) DO
        # Get the date at the end of that month.
        WITH ( DATETIME endOfMonth
                = CompletePartialDate ( @"Max DayOfMonth" , date ) ) DO
            # Find the ratio of the CBRFC forecast volume to the computed forecast over this month.
            WITH ( NUMERIC ratio
                = CBRFC . ( MapForecastLocationToCode ( forecastLocation ) CONCAT CBRFC Forecast Type ( ) ) [ endOfMonth ]
                    / SumFlowsToVolume ( InflowDisagg . ( "InflowForecastUnscaled" CONCAT forecastLocation ) ,
                        CompletePartialDate ( @"Min DayOfMonth" , date ) ,
                        endOfMonth ) ) DO
                # Set the forecast by multiplying the unscaled forecast by the scaling factor.
                InflowDisagg . ( "InflowForecast" CONCAT forecastLocation ) [ date ]
                    = InflowDisagg . ( "InflowForecastUnscaled" CONCAT forecastLocation ) [ date ] * ratio
            END WITH
        END WITH
    END FOR
END FOR
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00 AND @"t" == @"Start Timestep" AND SWITCHES.MODE [ ] == 0.00
```

Rule: 1 Cycle - Write Forecast as Average of Nearest Years
Priority: 87
Description:
Notes:

```
FOR ( STRING forecastLocation IN { "Vallecito" , "Lemon" , "Animas" , "Navajo" } ) DO
    # Loop over all daily timesteps
    FOR ( DATETIME date IN @"Start Timestep" TO @"Finish Timestep" ) DO
        # Get the list of nearest years
        WITH ( LIST NearestYears
            = FOR ( NUMERIC num IN GetNumbers ( 1.00 , InflowDisagg.NumberOfSimilarYears [ ] , 1.00 ) ) WITH LIST result = { } DO
                result
                = APPEND InflowDisagg . ( "HistoricalYearsSelected" CONCAT forecastLocation ) [ CompletePartialDate ( @"24:00:00 Max DayOfMonth" , date ) , num - 1.00 ] ONTO result
            END FOR ) DO
```

```
            # Average the flows for this day (i.e. Aug 13) using N values from the historical years; e.g. Aug 13,
            # 1983, Aug 13 1989, Aug 13, 2015.
            InflowDisagg . ( "InflowForecastUnscaled" CONCAT forecastLocation ) [ date ]
                = FOR ( NUMERIC year IN NearestYears ) AVE
                    HISTORIC . ( forecastLocation CONCAT "DailyInflow" ) [ CompletePartialDate
                                                                            ( NumberToYear ,
                                                                                ( year )
                                                                            IF ( # Need special logic on leap day
                                                                                 date == @"February 29" ) THEN
                                                                                date
                                                                                - 1.00 "day"
                                                                            ELSE
                                                                                date
                                                                            END IF ) ]
                END FOR
        END WITH
    END FOR
END FOR
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00 AND @"t" == @"Start Timestep" AND SWITCHES.MODE [ ] == 0.00
```

Rule: 1 Cycle - Find Nearest Years
Priority: 88
Description:
Notes:

```
FOR ( STRING forecastLocation IN { "Vallecito" ,
                                   "Lemon" ,
                                   "Animas" ,
                                   "Navajo" } ) DO
    FOR ( DATETIME givenMonth IN # Get a List of all the months in the run
                                 # {Aug 31, 2014, Sep 30, 2014,...}
                                 GetDates ( OffsetDate ( @"Start Timestep - 1" ,
                                                         1.00 ,
                                                         "1 Months" ) ,
                                            @"Finish Timestep" ,
                                            "1 Months" ) ) DO
        WITH ( LIST MonthsAndYearsForThisMonth
                 = # Get a list of all of the months that match the given Month
                   # {Aug 31, 2015, 2015}
                   FOR ( DATETIME month IN GetDates ( @"24:00:00 October 31, 1980" ,
                                                      @"24:00:00 September 30, 2020" ,
                                                      "1 Months" ) ) WITH LIST result = { } DO
                       result
                       = IF ( GetMonth ( month ) == GetMonth ( givenMonth ) ) THEN
                             APPEND { month ,
                                      GetYear ( month ) } ONTO result
                         ELSE
                             result
                         END IF
                   END FOR ) DO
            # Get a sorted list of the forecast location volume and corresponding year
            # {{1000AF, 2013} , {1200AF, 1985}, ...}
            WITH ( LIST SortedListOfYearsVolumes
                     = Sort ( FOR ( LIST monthYear IN MonthsAndYearsForThisMonth ) WITH LIST result = { } DO
                                result
                                = APPEND { HISTORIC . ( forecastLocation CONCAT "MonthlyInflow" ) [ monthYear < 0 > ] ,
                                           monthYear < 1 > } ONTO result
                              END FOR ) ) DO
                # Find the N nearest years based on the above list and the forecast volume.
                WITH ( LIST nearestYears ) DO
```

```
                    = GetNNearestYears ( SortedListOfYearsVolumes ,
                                         givenMonth ,
                                         forecastLocation ,
                                         CBRFC Forecast Type ( ) )
                FOR ( NUMERIC num IN GetNumbers ( 1.00 ,
                                                  InflowDisagg.NumberOfSimilarYears [ ] ,
                                                  1.00 ) ) DO
                    # Loop over the number of nearest years and set the nearest years (as numbers) to an Agg slot
                    InflowDisagg . ( "HistoricalYearsSelected" CONCAT forecastLocation ) [ givenMonth ,
                                                                                          num - 1.00 ]
                        = nearestYears 〈 num - 1.00 〉
                END FOR
            END WITH
        END WITH
    END WITH
  END FOR
END FOR
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00 AND @"t" == @"Start Timestep" AND SWITCHES.MODE [ ] == 0.00
```

Policy Group: 1 Cycle - MRM INFLOW FORECASTS
Description:
Notes:

Rule: Set SJC Diversion
Priority: 89
Description: Daily SJC diversion is directly related to the daily MUI flows. This correlation was made by comparing historic Azotea flows and MUI flows from 10/1/1980 - 8/31/2018. We are assuming on a daily basis that Azotea will pull whatever is available.
Notes:

```
FOR ( DATETIME date IN @"Start Timestep" TO @"Finish Timestep" ) DO
    San Juan Chama Project.Diversion Requested [ date ] = Max ( Min ( ( NavajoData.CBRFC Inflow [ date ] - 419.40 "cfs" ) * 1.00 "cfs" , 950.00 "cfs" ) / 6.61 "cfs" , 0.00 "cfs" )
END FOR
```

Execute Only When:

```
@"t" == @"Start Timestep" AND GetRunCycleIndex ( ) == 1.00 AND SWITCHES.MODE [ ] == 1.00
```

Rule: Set Inflows
Priority: 90
Description: Do not use execution constraints. Inflows must be input every cycle (why?) or model will not execute completely.
Notes:

```
FOR ( DATETIME date IN @"Start Timestep" TO @"Finish Timestep" ) DO
    Vallecito Reservoir.Inflow [ date ] = MRM Data.TraceVCRC2 [ date ]
    NavajoData.CBRFC Inflow [ date ] = MRM Data.TraceNVRN5 [ date ]
    Lemon Reservoir.Inflow [ date ] = MRM Data.TraceLEMC2 [ date ]
    Animas abv Nighthorse.Inflow [ date ] = MRM Data.TraceDRGC2 [ date ]
END FOR
```

Execute Only When:

```
@"t" == @"Start Timestep" AND SWITCHES.MODE [ ] == 1.00
```

Policy Group: Archived Rules
Description:
Notes:

Rule: SHORTAGE - Reduced TBF through Release this year
Priority: 91
Description: Reduces the previously calculated release to meet min 500 cfs by some increment for the time period specified. First do NIIP to reduce them by a specific amount during high season. Then do rest from here during same timeframe (summer) when we aren't already at 250.
Notes:

```
Navajo Reservoir.Release [ ] = Max ( Navajo Reservoir.Release [ ] - Shortage.ReleaseReduction [ ] , 250.00 "cfs" )
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" )
AND GetRunCycleIndex ( ) == 3.00
AND ( @"t" > @"24:00:00 July 1, 2022"
      AND @"t" < @"24:00:00 December 31, 2022" )
AND SWITCHES.Shortage [ ] == 1.00
```

Rule: SHORTAGE - Reduced TBF through Release next year
Priority: 92
Description: Reduces the previously calculated release to meet min 500 cfs by some increment for the time period specified. First do NIIP to reduce them by a specific amount during high season. Then do rest from here during same timeframe (summer) when we aren't already at 250.
Notes:

```
Navajo Reservoir.Release [ ] = Max ( Navajo Reservoir.Release [ ] - Shortage.ReleaseReduction_nextyear [ ] , 250.00 "cfs" )
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" )
AND GetRunCycleIndex ( ) == 3.00
AND ( @"t" > @"24:00:00 December 31, 2022"
      AND @"t" < @"24:00:00 December 31, 2023" )
AND SWITCHES.Shortage [ ] == 1.00
```

Rule: DROA 2022 Risk Analysis Contribution Baseflows
Priority: 93
Description: For DROA scenarios. Modify with scenario. This is for baseflows.
Notes:

```
Navajo Reservoir.Release [ ] = Navajo Reservoir.Release [ ] + 407.00 "cfs"
```

Execute Only When:

```
NOT HasRuleFiredSuccessfully ( "ThisRule" )
AND GetRunCycleIndex ( ) == 3.00
AND @"t" < @"24:00:00 January 1, 2023"
AND @"t" > @"24:00:00 October 31, 2022"
```

Rule: Set NIIP Diversions -MANUAL SHORTED next CY
Priority: 94
Description: Reduces NIIP diversion during a shortage in the timeframe specified. Use proportional estimates to reduce NIIP by an exact amount. THen resolve the remainder through the downstream release.
Notes:

```
NIIP.Diversion Requested [ ] = Max ( NIIP.StaticYearlyNIIPvalues [ ] - Shortage.NIIP Reduction_next year [ ] , 0.00 "cfs" )
```

Execute Only When:

```
GetRunCycleIndex ( ) == 1.00 AND ( @"t" > @"24:00:00 January 1, 2023" AND @"t" < @"24:00:00 December 31, 2023" ) AND SWITCHES.Shortage [ ] == 1.00
```

Rule: RecordShortageAndDemands
Priority: 95
Description: Whole demands from Cycle 3 is the volumetric addition of Navajo Release, NIIP, and San Juan-Chama, MINUS the 250cfs minimum daily release from Navajo in acre-feet. This ensures we are adding together only volumes that are available to short.
Notes:

```
ShortageData.ShortageVolumeToRecover [ ] = FlowToVolume ( Shortage.Local Inflow [ ] , @"t" )
ShortageData.WholeDemandsFromCycle3 [ ]
    = FlowToVolume ( Navajo Reservoir.Release [ ] ,
                     @"t" )
      + FlowToVolume ( NIIP.Diversion [ ] ,
                       @"t" )
      + FlowToVolume ( San Juan Chama Project.Diversion [ ] ,
                       @"t" )
      - 495.87 "acre-feet"
ShortageData.NavajoReleasesCycle3 [ ] = Navajo Reservoir.Release [ ]
```

Execute Only When:

```
GetRunCycleIndex ( ) == 3.00 AND SWITCHES.Magic Water [ ] == 1.00
```

Rule: SetSPRbyPeakDays
Priority: 96
Description: When SPR has begun (when very close to SPR time) , the SPR must be manually input here-- this rule should be turned off and the SPR being used should be input into the "NavajoData.KnownReleasePattern" slot.
Notes:

```
NavajoData.Cycle3SpringPeakRelease [ ] = IF ( SpringPeakReleaseSeason ( ) ) THEN
                                            NavajoData.SPRTables_7dayramp [ @"t" , NavajoData.Cycle3MaxDaysAtPeak [ @"t" ] ]
                                        END IF
```

Execute Only When:

```
GetRunCycleIndex ( ) == 3.00 AND NOT HasRuleFiredSuccessfully ( "ThisRule" )
```

Utility Group: Vallecito Functions
Description:
Notes:

Function: Winter Storage Season
Return Type: BOOLEAN
Arguments: ( )
Description:
Notes:

```
@"t" > @"October 31" OR @"t" < @"March 1"
```

Function: Vallecito Irrigation Season
Return Type: BOOLEAN
Arguments: ( )
Description:
Notes:

```
@"t" <= @"October 31" AND @"t" > @"May 9"
```

Function: DaysUntilEndOfWinterStorage
Return Type: NUMERIC
Arguments: ( )
Description: This function calculates the number of days from the current timestep to March 1st (end of winter storage season) of the current Water Year.
Notes:

```
IF ( Winter Storage Season ( ) AND @"t" > @"October 31" ) THEN
    GetJulianDate ( @"24:00:00 March 1, Next Year" ) - GetJulianDate ( @"t" )
ELSE
    IF ( Winter Storage Season ( ) AND @"t" < @"March 1" ) THEN
        GetJulianDate ( @"24:00:00 March 1, Current Year" ) - GetJulianDate ( @"t" )
    ELSE
        0.00 "day"
    END IF
END IF
```

Function: Remaining Winter Inflow Volume
Return Type: NUMERIC
Arguments: (OBJECT res)
Description:
Notes:

```
IF ( Winter Storage Season ( ) AND @"t" > @"October 31" ) THEN
    SumFlowsToVolume ( res . "Inflow" , @"t" , @"24:00:00 February Max DayOfMonth, Next Year" )
ELSE
    IF ( Winter Storage Season ( ) AND @"t" < @"March 1" ) THEN
        SumFlowsToVolume ( res . "Inflow" , @"t" , @"24:00:00 February Max DayOfMonth, Current Year" )
    ELSE
        0.00 "acre-feet"
    END IF
END IF
```

Function: Remaining Post Peak Inflow Volume
Return Type: NUMERIC
Arguments: ( )
Description:
Notes:

```
IF ( PostPeakSmoothingPeriod ( ) ) THEN
    SumFlowsToVolume ( Vallecito Reservoir.Inflow , @"t" , @"24:00:00 October Max DayOfMonth, Current Year" )
ELSE
    0.00 "acre-feet"
END IF
```

Function: PostPeakSmoothingPeriod
Return Type: BOOLEAN
Arguments: ( )
Description:
Notes:

```
@"t" >= NumberToDate ( VallecitoData.PostPeakSmoothingBeginDate [ ] ) AND @"t" < @"November 1"
```

Utility Group: Lemon Functions
Description:
Notes:

Function: Lemon Irrigation Season
Return Type: BOOLEAN
Arguments: ( )
Description:
Notes:

```
@"t" <= @"October 31" AND @"t" > @"April 30"
```

Function: LemonPondFillingSeason
Return Type: BOOLEAN
Arguments: ( )
Description: John Set the dates Sept 2021 as Sept 27 - Oct 1 for 500 af which is about 50 cfs per day for those five days.
Notes:

```
@"t" <= @"October 1" AND @"t" > @"September 26"
```

Utility Group: Nighthorse Functions
Description:
Notes:

Function: PickStepPump
Return Type: NUMERIC
Arguments: ( )
Description: This function returns the closest step pump to the value that was calculated in PumpingPerDay
Notes:

```
TableLookupDefaultTol ( NighthorseData.StepPumps , 0.00 , 0.00 , CalculateDailyFlow ( ) , @"t" , TRUE )
```

Function: ALP Pumping Season
Return Type: BOOLEAN
Arguments: ( )
Description:
Notes:

```
@"t" <= @"June 30" AND @"t" > @"April 30"
```

Function: CalculateDailyFlow
Return Type: NUMERIC
Arguments: ( )
Description:
Notes:

```
VolumeToFlow ( CalculateDailyVolumeToBePumped ( ) , @"t" )
```

Function: CalculateDailyVolumeToBePumped

Return Type: NUMERIC
Arguments: ( )
Description: 800 acrefeet denotes avg evap in May-June.
Notes:

$$\frac{\left(\text{NighthorseData.MaxStorage} [\,] + 800.00 \text{ "acre-feet"} - \text{Nighthorse Reservoir.Storage} [\,@\text{"t"}\,] \right)}{\left(\text{NighthorseData.RequestedPumpingPlan} [\,@\text{"t"}\,] - \frac{\text{NighthorseData.RequestedPumpingPlan} [\,@\text{"t"}\,]}{30.00} * \text{NighthorseData.PumpDaysOffPerMonth} [\,] \right)}$$

Function: IsFirstDayOfPumping
Return Type: BOOLEAN
Arguments: ( )
Description: UPDATE THIS to directly pull from input data!
Notes:

```
NighthorseData.RequestedPumpingPlan [ @"t - 1" ] == 0.00 AND NighthorseData.RequestedPumpingPlan [ @"t" ] > 0.00
```

Utility Group: Navajo Functions
Description:
Notes:

Function: SpringPeakReleaseSeason
Return Type: BOOLEAN
Arguments: ( )
Description:
Notes:

```
@"t" <= @"July 20" AND @"t" >= @"March 1"
```

Function: SummerBaseflowSeason
Return Type: BOOLEAN
Arguments: ( )
Description: Should be Sptember 30th but trying to avoid spikes.
Notes:

```
@"t" <= @"September 30" AND @"t" >= @"June 1"
```

Function: OLD SPROperationalSpillVolume
Return Type: NUMERIC
Arguments: ( )
Description: 374,777 af refers to the volume of the maximum hydrograph minus the base release of 350 cfs over the duration of the maximum SPR.
Notes:

```
NavajoData.Cycle3AvailableWater [ ] - 374,777.00 "acre-ft"
```

Function: SPROperationalSpillVolume
Return Type: NUMERIC
Arguments: ( )
Description: 625702 af refers to the volume of the maximum hydrograph (60-days plus ramps) minus the base release of 350 cfs over the duration of the maximum SPR.
Notes:

```
Max ( NavajoData.Cycle3AvailableWater [ ] - 625,702.00 "acre-feet" , 0.00 "acre-ft" )
```

Function: VolumeBeyondNose
Return Type: NUMERIC
Arguments: ( )
Description: See if there's any extra spill that couldn't be taken care of with SPR Nose. This is done by taking the maximum release through march 1 (5,000 cfs) and multiplying it by 57 days (first day of SPR is April 27.
Notes:

```
OLD SPROperationalSpillVolume ( ) - 565,269.00 "acre-ft"
```

Function: SpikeSeason
Return Type: BOOLEAN
Arguments: ( )
Description: Use later date (Sept 25-Oct 5) if min, or expecting small spike.
Use earlier date (Sept 10-15 start) if max, or expecting large spike.
Notes:

```
@"t" <= @"September 30" AND @"t" >= @"September 15"
```

Function: FirstAWCalcDate
Return Type: DATETIME
Arguments: ( )
Description:
Notes:

```
DateMax ( @"24:00:00 March 1, Current Year" , @"Start Timestep" )
```

Function: USACEWinterLimitSeason
Return Type: BOOLEAN
Arguments: ( )
Description:
Notes:

```
@"t" >= @"January 1" AND @"t" <= @"July 15"
```

Function: MinReleaseFunction
Return Type: NUMERIC
Arguments: ( )
Description:
Notes:

```
IF ( Navajo Reservoir.Pool Elevation [ @"t - 1" ] <= 6,010.00 "ft" ) THEN
    249.00 "cfs"
ELSE
    IF ( Navajo Reservoir.Pool Elevation [ @"t - 1" ] > 6,060.00 "ft" ) THEN
        349.00 "cfs"
    ELSE
        299.00 "cfs"
    END IF
END IF
```

Function: MinReleaseFunctionbyFcst
Return Type: NUMERIC
Arguments: ( )
Description: 1=249
3=449
else = 299
Notes:

```
IF ( SWITCHES.CBRFC Forecast Type [ ] == 1.00 ) THEN
    299.00 "cfs"
ELSE
    IF ( SWITCHES.CBRFC Forecast Type [ ] == 3.00 ) THEN
        399.00 "cfs"
    ELSE
        299.00 "cfs"
    END IF
END IF
```

Function: MaxReleaseFunction_SafetyFactor
Return Type: NUMERIC
Arguments: ( )
Description:
Notes:

```
GetMaxReleaseGivenInflow ( Navajo Reservoir , Navajo Reservoir.Inflow [ ] , @"t" ) * 1.00
```

Function: OutflowRequredToMeetTPE1
Return Type: NUMERIC
Arguments: (OBJECT Res, SLOT ResInflow, SLOT ResStorage, SLOT ResEOYMinStorage)
Description:
Notes:

```
IF ( @"t" == @"Finish Timestep" ) THEN
    ResInflow [ @"t - 1" ]
```

```
ELSE
    TargetHWGivenInflow ( Res ,
                          @"t" ,
                          DateMin ( NextDate ( @"t" , @"September 30" ) , @"Finish Timestep" ) ,
                          StorageToElevation ( Res , ResEOYMinStorage [ ] ) ,
                          SumFlowsToVolume ( ResInflow , @"t" , DateMin ( NextDate ( @"t" , @"September 30" ) , @"Finish Timestep" ) ) ,
                          ResStorage [ @"t - 1" ] )
END IF
```

Function: NavajoFloodControlUpperColumnValue
Return Type: NUMERIC
Arguments: ( )
Description:
Notes:

```
NavajoData.FloodControlTables [ @"t" , Ceiling ( SumFlowsToVolume ( Navajo Reservoir.Inflow , @"t" , @"24:00:00 July 15, Current Year" ) , 100,000.00 "acre-ft" ) ]
```

Function: NavajoFloodControlLowerColumnValue
Return Type: NUMERIC
Arguments: ( )
Description:
Notes:

```
NavajoData.FloodControlTables [ @"t" , Floor ( SumFlowsToVolume ( Navajo Reservoir.Inflow , @"t" , @"24:00:00 July 15, Current Year" ) , 100,000.00 "acre-ft" ) ]
```

Utility Group: Global Functions
Description:
Notes:

Function: Remaining Spring Runoff Volume
Return Type: NUMERIC
Arguments: (OBJECT res)
Description:
Notes:

```
IF ( @"t" >= @"March 1" AND @"t" <= @"July 31" ) THEN
    SumFlowsToVolume ( res . "Inflow" , @"t" , @"24:00:00 July 31, Current Year" )
ELSE
    0.00 "acre-feet"
END IF
```

Function: CalculateAvailableIrrigationVolume
Return Type: NUMERIC
Arguments: (OBJECT Res, SLOT ResInflow, SLOT ResStorage, SLOT ResEOYMinStorage, SLOT ResEvap)
Description:
Notes:

```
( FlowToVolume ( OutflowRequredToMeetTPE ( Res , ResInflow , ResStorage , ResEOYMinStorage ) , @"t" ) - ResEvap [ @"t - 1" ] ) * LENGTH @"t" TO DateMin ( NextDate ( @"t" , @"November 1" ) , @"Finish Timestep" )
```

Function: OutflowRequredToMeetTPE
Return Type: NUMERIC
Arguments: (OBJECT Res, SLOT ResInflow, SLOT ResStorage, SLOT ResEOYMinStorage)
Description:
Notes:

```
IF ( @"t" == @"Finish Timestep" ) THEN
```

```
    ResInflow [ @"t - 1" ]
ELSE
    TargetHWGivenInflow ( Res ,
                          @"t" ,
                          DateMin ( NextDate ( @"t" ,
                                               @"November 1" ) ,
                                    @"Finish Timestep" ) ,
                          StorageToElevation ( Res ,
                                               ResEOYMinStorage [ ] ) ,
                          SumFlowsToVolume ( ResInflow ,
                                             @"t" ,
                                             DateMin ( NextDate ( @"t" ,
                                                                  @"November 1" ) ,
                                                       @"Finish Timestep" ) ) ,
                          ResStorage [ @"t - 1" ] )
END IF
```

Function: Adjust GainLoss
Return Type: NUMERIC
Arguments: (OBJECT Reach)
Description:
Notes:

```
FOR ( NUMERIC i IN { 0.00 , 1.00 , 2.00 , 3.00 , 4.00 , 5.00 , 6.00 , 7.00 , 8.00 } ) WITH NUMERIC exceedance = 0.10 DO
    exceedance = IF ( IsNaN Reach . "Inflow" [ @"t" ] ) THEN
                     0.50
                 ELSE
                     IF ( Reach . "Inflow" [ @"t" ] > GainLossCalculations.Table [ STRINGIFY Reach , i ] ) THEN
                         Min ( 0.90 , exceedance + 0.10 )
                     ELSE
                         exceedance
                     END IF
                 END IF
END FOR
```

Utility Group: Nearest Years
Description:
Notes:

Function: GetNNearestYears
Return Type: LIST
Arguments: (LIST sortedListOfYearsValues, DATETIME month, STRING forecastLocation, STRING forecastType)
Description:
Notes:

```
WITH SLOT CBRFCSlotName = CBRFC . ( MapForecastLocationToCode ( forecastLocation ) CONCAT forecastType ) DO
    WITH LIST NumSimilarYears = GetNumbers ( 0.00 ,
                                             InflowDisagg.NumberOfSimilarYears [ ]
                                             - 1.00 ,
                                             1.00 ) DO
        IF ( CBRFCSlotName [ month ] < sortedListOfYearsValues 〈 0 〉 〈 0 〉 ) THEN
            # The CBRFC volume is lower than any years in history. Use the lowest N years.
            FOR ( NUMERIC index IN NumSimilarYears ) WITH LIST result = { } DO
                result
                = APPEND sortedListOfYearsValues 〈 index 〉 〈 1 〉 ONTO result
            END FOR
        ELSE IF ( CBRFCSlotName [ month ] > sortedListOfYearsValues 〈 ( LENGTH sortedListOfYearsValues )
                                                                    - 1.00 〉 〈 0 〉 ) THEN
            # The CBRFC volume is higher than any year in history. Use the highest N years.
            FOR ( NUMERIC index IN NumSimilarYears ) WITH LIST result = { } DO
                result
                = APPEND sortedListOfYearsValues 〈 ( ( LENGTH sortedListOfYearsValues )
                                                     - 1.00 )
                                                     - index 〉 〈 1 〉 ONTO result
```

```
            END FOR
        ELSE
            # The CBRFC volume is in the historical record. Find the year that is lower than the CBRFC volume.
            # Get the two years on either side of this year.
            FOR ( NUMERIC num IN GetNumbers ( 0.00 ,
                                              ( LENGTH sortedListOfYearsValues )
                                              - 1.00 ,
                                              1.00 ) ) WITH LIST result = { } DO
                result
                = IF ( ( sortedListOfYearsValues < num > < 0 > ) < CBRFCSlotName [ month ]
                       AND CBRFCSlotName [ month ] <= sortedListOfYearsValues < Min ( num
                                                                                      + 1.00 ,
                                                                                      ( LENGTH sortedListOfYearsValues )
                                                                                      - 1.00 ) > < 0 > ) THEN
                    FOR ( NUMERIC index IN NumSimilarYears ) WITH LIST finalResult = { } DO
                        finalResult
                        = APPEND sortedListOfYearsValues < Min ( num
                                                                 + index ,
                                                                 ( LENGTH sortedListOfYearsValues )
                                                                 - 1.00 ) > < 1 > ONTO finalResult
                    END FOR
                ELSE
                    result
                END IF
            END FOR
        END IF
    END WITH
END WITH
```

Function: MapForecastLocationToCode
Return Type: STRING
Arguments: (STRING forecastLocation)
Description:
Notes:

```
IF ( forecastLocation == "Vallecito" ) THEN
    "VCRC2"
ELSE IF ( forecastLocation == "Lemon" ) THEN
    "LEMC2"
ELSE IF ( forecastLocation == "Animas" ) THEN
    "DRGC2"
ELSE
    "NVRN5"
END IF
```

Function: CBRFC Forecast Type
Return Type: STRING
Arguments: ( )
Description:
Notes:

```
WITH NUMERIC type = SWITCHES.CBRFC Forecast Type [ ] DO
    IF ( type == 2.00 ) THEN
        "most"
    ELSE IF ( type == 1.00 ) THEN
        "min"
    ELSE
        # type == 3
        "max"
    END IF
END WITH
```

Utility Group: Loss Functions

Description:
Notes:

Function: Lookup Loss Percentile
Return Type: NUMERIC
Arguments: (STRING forecastLocation)
Description:
Notes:

```
TableInterpolation ( GainLossCalculations.ForecastLookupTable ,
                     0.00 ,
                     1.00 ,
                     ( ForecastVolume ( forecastLocation ) / HISTORIC.AverageInflow [ @"t" , GetReservoirFromForecastLocation ( forecastLocation ) ] ) ,
                     @"t" )
```

Function: ForecastVolume
Return Type: NUMERIC
Arguments: (STRING forecastLocation)
Description:
Notes:

```
IF ( # 24 Mo Study Mode
     SWITCHES.MODE [ ] == 0.00 ) THEN
    CBRFC . forecastLocation CONCAT CBRFC Forecast Type ( ) [ @"24:00:00 Current Month Max DayOfMonth, Current Year" ]
ELSE IF ( # ESP MRM
          SWITCHES.MODE [ ] == 1.00 ) THEN
    SumFlowsToVolume ( MRM Data . "Trace" CONCAT forecastLocation ,
                       @"24:00:00 Current Month Min DayOfMonth, Current Year" ,
                       @"24:00:00 Current Month Max DayOfMonth, Current Year" )
ELSE
    STOP_RUN "No valid mode set"
END IF
```

Function: GetReservoirFromForecastLocation
Return Type: STRING
Arguments: (STRING forecastLocation)
Description:
Notes:

```
IF ( forecastLocation == "VCRC2" ) THEN
    "Vallecito Reservoir"
ELSE IF ( forecastLocation == "LEMC2" ) THEN
    "Lemon Reservoir"
ELSE IF ( forecastLocation == "DRGC2" ) THEN
    "Animas"
ELSE IF ( forecastLocation == "NVRN5" ) THEN
    "Navajo Reservoir"
ELSE
    STOP_RUN "Unexpected forecast location"
END IF
```